

Compound Figure Separation

Applied Deep Learning

Tobias Ponesch, 11818774

January 2026

1 Problem Statement & Motivation

During my bachelor thesis on extracting metadata from scientific charts, I encountered a significant bottleneck: the prevalence of **compound figures**. These are composite images that bundle multiple sub-figures or panels, such as charts, illustrations, or biomedical scans, into a single figure.

This project addresses the critical need for automated **compound figure separation**. Most downstream tasks, including **chart understanding**, **OCR**, **table extraction**, and **figure-type classification**, assume individual figures as input. Without robust separation, automation pipelines break at the first step, leaving scientific figures largely inaccessible for large-scale analytics.

While existing research heavily favors medical imagery, this project focuses on **general scientific figures**, bridging the gap with a dedicated dataset and a YOLO-based detection model capable of splitting complex figures into their individual components.

2 Relevant Scientific Literature

Most existing research focuses on **medical images** (X-rays, MRI). The standard dataset is the **Image-CLEF Medical dataset**. Approaches typically rely on domain-specific features or side loss [1, 2, 3] and do not generalize well to **general scientific figures** (charts, illustrations, tables).

Some research has explored general figures [4, 5], but publicly available implementations (e.g., *Compound Figure Separator*¹) often perform poorly on mixed chart types. Currently, there are **no publicly available datasets** dedicated to general scientific figure separation that include bounding box annotations for complex layouts.

3 Methodology

This section describes the dataset creation pipeline used in this project. All datasets were created with the goal of enabling robust training and evaluation of compound figure separation models. The final datasets are publicly available and can be downloaded as multiple ZIP archives containing images and labels from the Hugging Face repository *CompoundFigureSeparation*.²

3.1 Overview

In total, three **model-ready datasets** were created, namely `04_all_classes`, `05_selected_classes` and `06_compound_chart_splitter`. Each dataset targets a different level of abstraction in the compound figure separation problem. The datasets were generated through a multi-stage pipeline combining real annotated figures and synthetically generated data.

3.2 Initial Dataset Analysis and Figure Selection

Several existing datasets were inspected, including DocFigure [6], FigureSeer [7], MedICaT [8], SciCap [9], and SCI-3000 [10]. Among these, only MedICaT contains explicitly annotated compound figures. However, upon inspection, the bounding box quality was deemed insufficient for reliable training.

¹<https://github.com/apple2373/figure-separator>

²<https://huggingface.co/datasets/TobiPoni/CompoundFigureSeparation>

As a result, approximately 9,500 figures were extracted directly from the SCI-3000 dataset. Custom Python scripts were developed to manually curate this corpus. Each extracted figure was labeled as either *usable* or *discarded*, and further categorized as either a **single figure** or a **compound figure**. Single figures were additionally labeled as **Chart**, **Illustration**, **Image**, **Table**, or **Other**.

This process resulted in: **1,145 labeled single figures** and **700 real compound figures**.

3.3 Manual Annotation of Real Compound Figures

All real compound figures were annotated using **Label Studio**[11]. Bounding boxes were created for the following classes: **Chart**, **Illustration**, **Image**, **Other**, **Table**, as well as structural sub-elements including **Shared Legend**, **Shared Title**, **Shared X-Axis**, **Shared Y-Axis**, and **Subpanel**. These annotations serve as the primary source of real-world ground truth data for validation and testing. An example of a real compound figure can be found as Figure 5 in the Appendix.

3.4 Synthetic Dataset Generation

The labeled single figures were used to generate **10,000 synthetically stitched compound figures**. These figures were created by programmatically assembling multiple single figures into compound layouts while preserving bounding box annotations.

During early training experiments, models consistently struggled to correctly identify chart sub-structures such as shared axes, legends, titles, and subpanels. To mitigate this limitation, an additional **2,500 synthetically generated compound charts** were created using a custom **Matplotlib**-based plot generator. The generator produces multi-panel figures with diverse grid layouts, varying panel configurations, and probabilistic inclusion of shared titles, legends, and axis-sharing modes. Different titling strategies (text-based and enumerated), placement schemes, font styles, and spacing parameters are sampled to increase layout variability. All components are annotated with pixel-accurate bounding boxes using **Matplotlib**’s rendering backend, enabling reliable **YOLO** label generation. This augmentation step proved essential to improve performance on rare structural classes.

3.5 First Model-Ready Dataset: 04_all_classes

The first model-ready dataset, **04_all_classes**, combines **10,000** stitched synthetic compound figures, **2,500** synthetic compound charts, and **700** real compound figures.

The dataset is split into training, validation, and test sets. The training split contains synthetic and real data, while validation and test splits contain only real compound figures to ensure strict domain evaluation.

3.5.1 Dataset Split Evolution

V1: Baseline Sim2Real The initial version used a random split (70% train, 15% validation, 15% test). This resulted in severe class imbalance, especially for shared elements (classes 4–7) and tables, causing the model to ignore rare classes entirely.

V2: Stratified Split and Oversampling To address this issue, stratified splitting was introduced alongside aggressive oversampling, applying a **25× factor** to shared elements and a **5× factor** to subpanels. In addition, synthetic tables were injected into the validation and test splits to prevent empty-class partitions. While these measures significantly improved training stability, the scarcity of real samples containing shared structural elements remained a limiting factor.

V3: Plot-Augmented Dataset The final version introduces plot-focused synthetic compound charts containing explicit shared annotations. **Validation size was increased to 25%** to surface rare-class errors earlier. This version significantly improved recall on shared elements. Figure 1 shows the class distribution of the **04_all_classes** dataset (V3) across the training, validation, and test splits. While the training split is dominated by synthetically generated samples, the validation and test splits contain only real compound figures, resulting in significantly fewer but more realistic samples for rare structural classes.

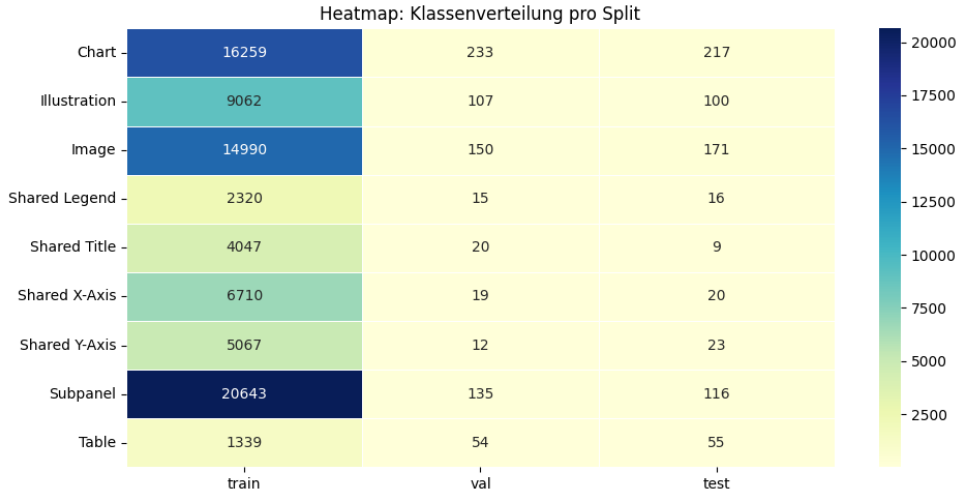


Figure 1: Class distribution across train, validation, and test splits for the V3 version of `04_all_classes`. Validation and test contain only real data, highlighting the scarcity of rare structural classes.

3.5.2 Key Insight

When real data is scarce, synthetic augmentation is not optional but essential. A model cannot learn structures it never observes.

3.6 Second Model-Ready Dataset: `05_selected_classes`

To decouple high-level figure detection from fine-grained sub-structure detection, a second dataset was created. `05_selected_classes` contains the same images as `04_all_classes`, but only retains the classes `Chart`, `Illustration`, `Image`, `Other`, and `Table` (remapped). All shared elements and subpanels were removed from the labels. This dataset enables training a model focused purely on separating compound figures into their major components. Figure 2 illustrates the class distribution of the `05_selected_classes` dataset across training, validation, and test splits. By restricting labels to high-level figure categories, the dataset exhibits a more balanced distribution in the validation and test splits compared to the full multi-class setting.

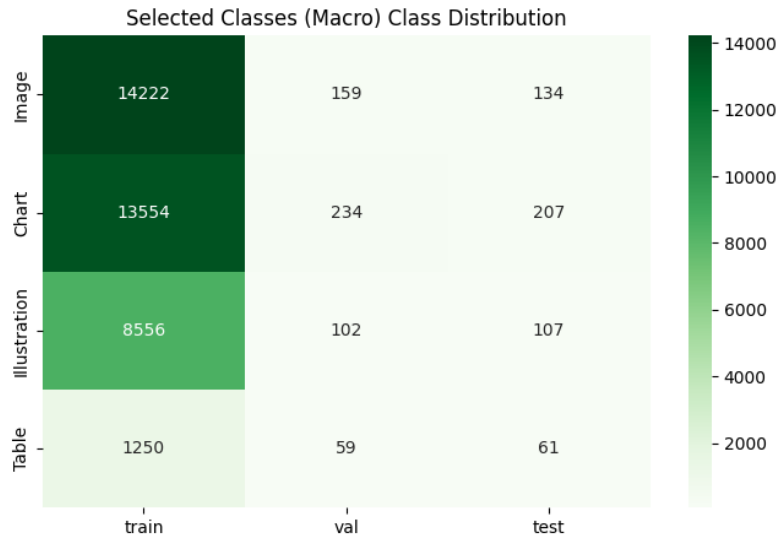


Figure 2: Class distribution across dataset splits for `05_selected_classes`, showing a more balanced validation and test distribution after restricting labels to high-level classes.

3.7 Third Model-Ready Dataset: 06_compound_chart_splitter

The final dataset focuses exclusively on compound charts. It contains only images composed entirely of charts and excludes illustrations, tables, and other image types. Labels are restricted to the structural classes **Shared Legend**, **Shared Title**, **Shared X-Axis**, **Shared Y-Axis**, and **Subpanel**. All labels were remapped to a contiguous class range. Due to the limited number of hand-labeled real compound charts, aggressive validation and test splits of 30% and 20% were used to expose generalization issues early during training.

3.8 Dataset Structure and Availability

All datasets follow a consistent **YOLO-compatible directory structure** and contain the required components for direct model training, including the image files, corresponding YOLO label files, a **data.yaml** configuration file, and explicit **train.txt**, **val.txt**, and **test.txt** split definitions. This structure ensures reproducibility and allows seamless integration with common YOLO training pipelines.

In addition to the model-ready datasets, several **pre-datasets** are included to fully document and reproduce the dataset creation process. The directory **01_raw** contains the unmodified figures extracted directly from the SCI-3000 PDFs, representing the initial input to the pipeline. The directory **02_assets** stores manually curated and labeled single figures, which were classified as **Chart**, **Illustration**, **Image**, **Table**, or **Other**, as well as auxiliary assets used for synthetic data generation. The directory **03_intermediate** contains intermediate products of the pipeline, including hand-labeled real compound figures, synthetically stitched compound figures, and synthetically generated compound charts with complex shared structures.

By exposing both intermediate and final artifacts, the dataset release enables full transparency and reproducibility, as well as the possibility to extend on this dataset and include more real compound figures.

3.8.1 Extending the Dataset

Despite the extensive use of synthetic augmentation, the primary limiting factor of the current datasets remains the **scarcity of hand-labeled real compound figures**. Even after curating and annotating the SCI-3000 corpus, the available ground truth covers only a fraction of the extracted figures. More than **6,000 additional figures** from SCI-3000 remain unlabeled and could substantially increase the diversity and realism of the dataset.

Labeling the remaining figures would allow the model to learn a broader range of layout patterns and shared structural elements directly from real data, which is expected to significantly improve generalization performance. Importantly, the entire dataset construction pipeline was designed to be **fully reusable**. The existing Python notebooks and scripts can be rerun to regenerate all model-ready datasets at larger scale, requiring only minor adjustments to **oversampling factors**, **stratification strategies**, and **train/validation/test split ratios**. This makes future dataset expansion straightforward and ensures that improvements in ground truth quality can be seamlessly propagated to all dataset variants.

4 Baseline Models

4.1 YOLO Background and Model Variants

YOLO (You Only Look Once) is a family of single-stage object detection models that predict bounding boxes and class probabilities in a single forward pass. Its end-to-end design enables efficient training and inference while maintaining competitive detection accuracy, making it well suited for large-scale dataset experimentation.

All experiments in this work were conducted using the **YOLOv11** architecture as provided by the Ultralytics framework (**ultralytics v8.3.241**). Three model sizes were evaluated to study the trade-off between model capacity, input resolution, and performance: **yolo11n** (nano), **yolo11s** (small), and **yolo11m** (medium). Models were trained on the three model-ready datasets introduced earlier: **04_all_classes**, **05_selected_classes**, and **06_compound_chart_splitter**.

4.2 Training Protocol and Evaluation

All models were initialized from pretrained YOLOv11 weights and trained using the default Ultralytics training pipeline unless stated otherwise. The **best checkpoint** for each run was selected based on the **validation** mAP_{50-95} and subsequently evaluated on the **test split**. Metrics reported in this section therefore reflect test-set performance of the validation-selected checkpoint. Qualitative examples (successful predictions and typical failure cases) are provided in the repository directory `docs/assets`^{3,4,5}.

4.3 Training Configurations

Table 1 summarizes the training configurations explored in this work. Input resolution, batch size, number of epochs, and model scale were varied to evaluate their impact on detection performance.

Table 1: Overview of trained YOLOv11 baseline configurations.

Dataset	Model	Image Size	Epochs	Batch
04.all_classes	yolo11n	640	20	32
04.all_classes	yolo11s	960	40	16
04.all_classes	yolo11m	1280	70	8
04.all_classes	yolo11m	1280	30	8
04.all_classes	yolo11s	1280	60	16
05.selected_classes	yolo11n	1280	50	16
06.compound_chart_splitter	yolo11n	1280	50	16
06.compound_chart_splitter	yolo11s	1280	50	16
06.compound_chart_splitter	yolo11m	1280	100	8

4.4 Quantitative Results

Table 2 reports the detection performance of all trained models using standard object detection metrics. The primary optimization target was mAP_{50-95} , evaluated on the test sets.

Table 2: Detection performance of trained YOLOv11 baselines including the main training configuration.

Dataset	Model	Img	Ep	Bs	mAP_{50-95}	mAP_{50}
04.all_classes	yolo11n	640	20	32	0.475	0.589
04.all_classes	yolo11s	960	40	16	0.589	0.652
04.all_classes	yolo11s	1280	60	16	0.559	0.640
05.selected_classes	yolo11n	1280	50	16	0.888	0.949
06.compound_chart_splitter	yolo11n	1280	50	16	0.436	0.622
06.compound_chart_splitter	yolo11s	1280	50	16	0.433	0.587
06.compound_chart_splitter	yolo11m	1280	100	8	0.436	0.598

4.5 Best Model per Dataset

Figure 3 compares the best-performing model for each dataset in terms of mAP_{50-95} , evaluated on the test sets. The 05.selected_classes dataset achieves substantially higher performance, indicating that restricting the task to high-level semantic classes simplifies the learning problem.

4.6 Per-Class Performance Analysis

Figure 4 shows the per-class mAP_{50-95} scores for the best model of each dataset, evaluated on the test set. While semantic classes such as **Chart**, **Image**, and **Table** achieve strong performance, structural classes, particularly **Shared Title** and **Shared Legend**, remain challenging due to limited real training data and high layout variability. A qualitative inspection of predictions is provided in the repository under `docs/assets/`, with examples for 04.all_classes³, 05.selected_classes⁴, and 06.compound_chart_splitter⁵.

³./docs/assets/04.all_classes_predictions

⁴./docs/assets/05.selected_classes_predictions

⁵./docs/assets/06.compound_chart_splitter_predictions

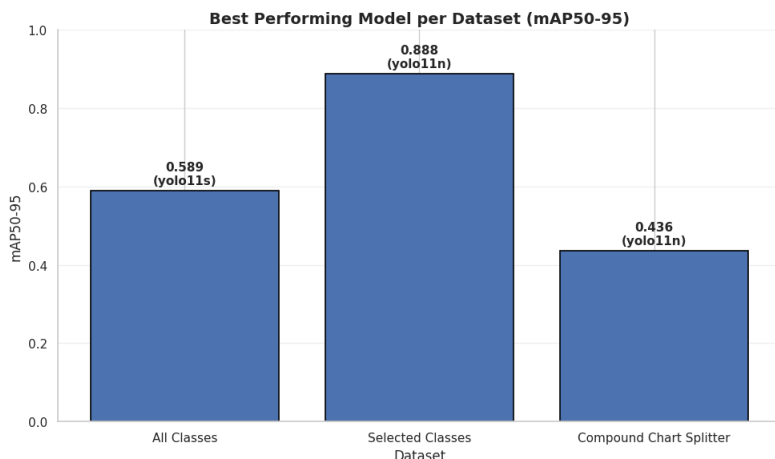


Figure 3: Best-performing model per dataset measured by mAP_{50-95} .



Figure 4: Per-class mAP_{50-95} for the best-performing model of each dataset.

4.7 Training Environment

All models were trained on a workstation equipped with an **NVIDIA GeForce RTX 3090** GPU (24 GB VRAM), an **AMD Ryzen 7 5800X** CPU (8 cores, 16 threads), and **32 GB DDR4 RAM**. The training environment used **Python 3.12**, **PyTorch 2.9**, **CUDA 12.8**, and **ultralytics v8.3.241**.

5 Conclusion

This project introduced a dataset creation pipeline and three YOLO-ready datasets (`04_all_classes`, `05_selected_classes`, and `06_compound_chart_splitter`) for **general scientific compound figure separation**. The experiments show that YOLOv11 is a strong baseline: high-level separation on `05_selected_classes` is largely solved ($mAP_{50-95} \approx 0.89$), while fine-grained structural detection in compound charts (`06_compound_chart_splitter`) remains challenging ($mAP_{50-95} \approx 0.44$), particularly for **Shared Title** and **Shared Legend**.

A key learning is the **data ceiling**: missing real-world variability cannot be compensated by simply training larger models for longer. While scaling capacity and training time can help to some extent, adding more **real annotated** compound figures is expected to yield larger gains than further model scaling.

Finally, this work served as a “bring your own data” reality check: most effort went into extraction, synthesis, and annotation, and it is hard to predict whether a new dataset will be broadly useful. Still, training baselines is essential to validate the dataset. In practice, **Label Studio** proved highly effective for annotation, and **YOLOv11** achieved strong results even with limited real training data.

A Appendix

A.1 Example plots of the intermediate data

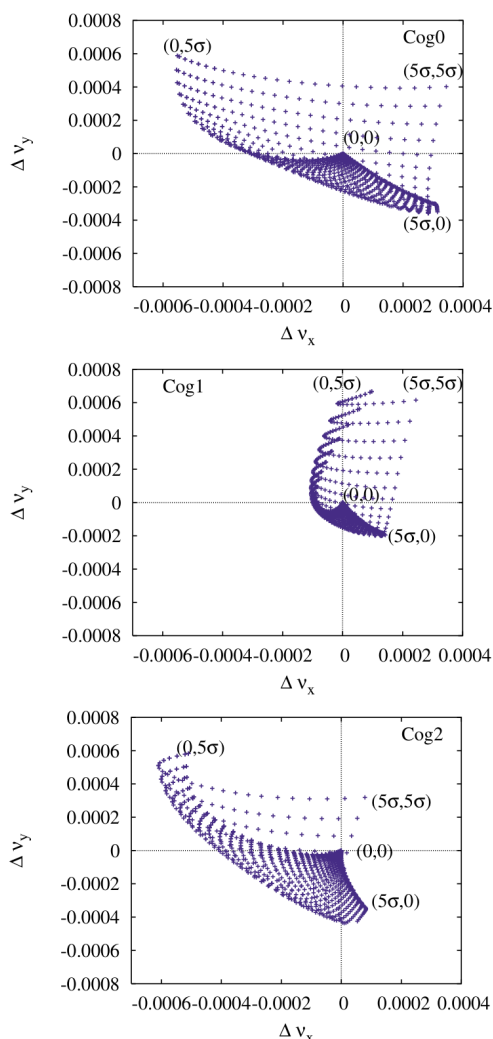


Figure 5: Example of a real compound figure from SCI-3000.

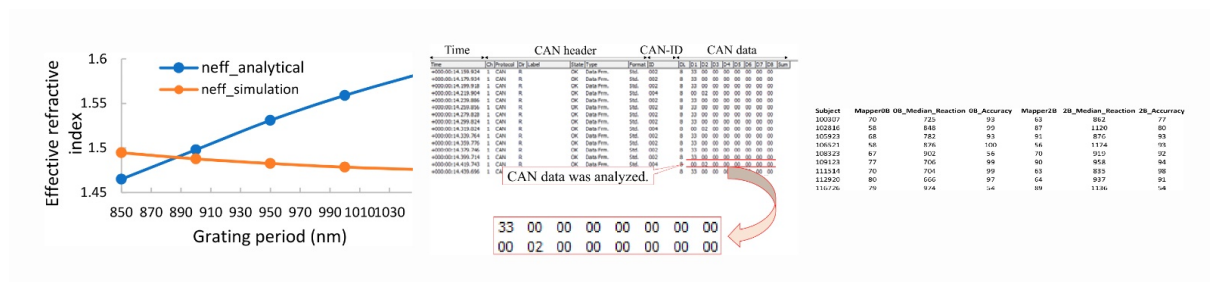


Figure 6: Example of a synthetically generated compound figure from SCI-3000 singles.

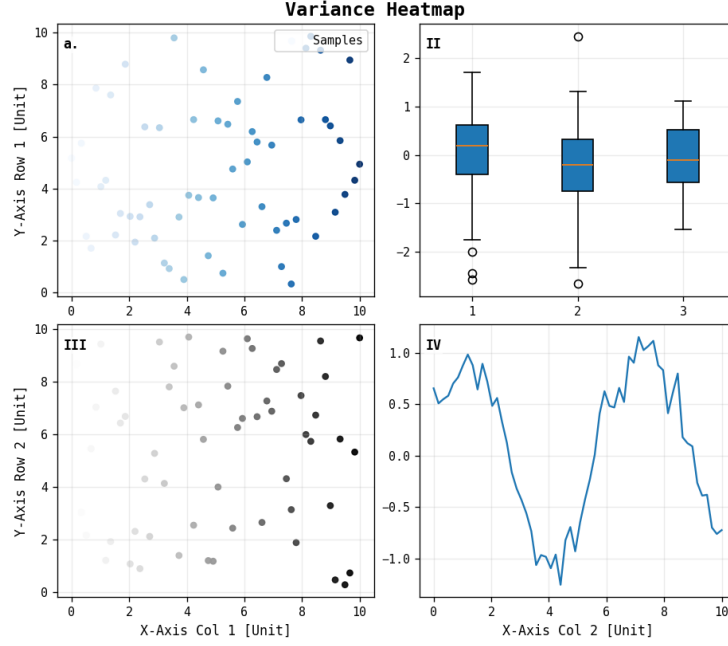


Figure 7: Example of a synthetically generated compound plot created with Matplotlib.

A.2 Time Tracking

References

- [1] Tianyuan Yao et al. “Compound Figure Separation of Biomedical Images with Side Loss”. In: *arXiv preprint arXiv:2107.08650* (2021). arXiv: [2107.08650](https://arxiv.org/abs/2107.08650) [cs.CV].
- [2] Tianyuan Yao et al. “Compound Figure Separation of Biomedical Images: Mining Large Datasets for Self-supervised Learning”. In: *arXiv preprint arXiv:2208.14357* (2022). arXiv: [2208.14357](https://arxiv.org/abs/2208.14357) [cs.CV].
- [3] Pengyuan Li et al. “Compound image segmentation of published biomedical figures”. In: *Bioinformatics* 34.14 (2018), pp. 2549–2557.
- [4] Satoshi Tsutsui and David Crandall. “A Data Driven Approach for Compound Figure Separation Using Convolutional Neural Networks”. In: *arXiv preprint arXiv:1703.05105* (2017). arXiv: [1703.05105](https://arxiv.org/abs/1703.05105) [cs.CV].
- [5] Noah Siegel et al. “FigureSeer: Parsing Result-Figures in Research Papers”. In: *European Conference on Computer Vision (ECCV) Workshops*. Springer, 2016.
- [6] *DocFigure: A Dataset for Scientific Document Figure Classification*. URL: <https://cvit.iiit.ac.in/usodi/Docfig.php>.
- [7] *FigureSeer: Parsing Result-Figures in Research Papers*. URL: <https://prior.allenai.org/projects/figureseer>.
- [8] Sanjay Subramanian et al. *MedICaT: A Dataset of Medical Images, Captions, and Textual References*. 2020. arXiv: [2010.06000](https://arxiv.org/abs/2010.06000) [cs.CV]. URL: <https://arxiv.org/abs/2010.06000>.
- [9] *SciCap: Scientific Figures Dataset*. URL: <https://github.com/tingyaohsu/SciCap>.
- [10] Filip Darmanovic. *SCI-3000: A novel dataset for the task of figure, table, and caption extraction from scientific PDFs*. 2020. URL: <https://repositum.tuwien.at/handle/20.500.12708/81300>.
- [11] Maxim Tkachenko et al. *Label Studio: Data labeling software*. Open source software available from <https://github.com/HumanSignal/label-studio>. 2020-2025. URL: <https://github.com/HumanSignal/label-studio>.

Task	What's included	Time (h)
Project setup	scope, goals, planning, repository initialization	4
Background research	related work, baseline selection, approach justification	4
Data acquisition	obtaining datasets/sources, organizing inputs, initial access checks	6
Data exploration	sampling, sanity checks, understanding distributions and edge cases	4
Data curation	filtering, cleaning, quality screening, removing unusable samples	4
Annotation planning	label schema, guidelines, class definitions, tooling decisions	3
Manual annotation	labeling work, review passes, consistency checks	20
Data preprocessing	format conversions, label normalization, path handling, remapping	5
Data augmentation / synthesis	generating additional data, variations, auto-label creation	10
Dataset construction	merging sources, creating dataset versions, class filtering	5
Split design	train/val/test policy, stratification, balancing/oversampling strategy	6
Validation & integrity checks	automated checks, spot checks, debugging broken samples	3
Training pipeline setup	scripts/configuration, environment preparation, reproducibility setup	5
Model training	baseline runs, experiment tracking, checkpoint handling	15
Debugging & iteration	fixing issues, re-generating data, re-labeling, reruns	6
Qualitative analysis	visual inspection, example selection, failure-case review	4
Demo / application	simple interface, inference pipeline, visualization	3
Documentation	README, methodology write-up, usage instructions	5
Final deliverables	report, presentation, submission preparation	6
Total		~120

Table 3: Task Breakdown and Time Allocation